Week 1 Assignment: Translating UML into Code

Master of Science

Information Technology

Technology Leadership

Gabriel Twizerimana

University of Denver College of Professional Studies

April 5, 2026

Faculty: Nathan Braun, MS

Dean: Bobbie Kite, PhD

Table of Contents

Object-Oriented University Parking System 1
Statement of the Problem and Requirements 1
UML Diagrams 1
Class Explanations and Implementation Notes 3
Core Domain Classes 3
Management Classes (The "Managers") 3
Controller/Command Classes 4
Survey of Unit Tests 4
Challenges and lessons learned 5
Challenges 5
Lessons Learned 5
Project Configuration (pom.xml) 6
Execution and Output 7
System Output: 7

Object-Oriented University Parking System

Statement of the Problem and Requirements

The modern urban environment requires an automated, reliable method for managing vehicle parking and associated financial transactions. This project addresses the need for a centralized **Parking Management System** capable of handling multiple administrative layers.

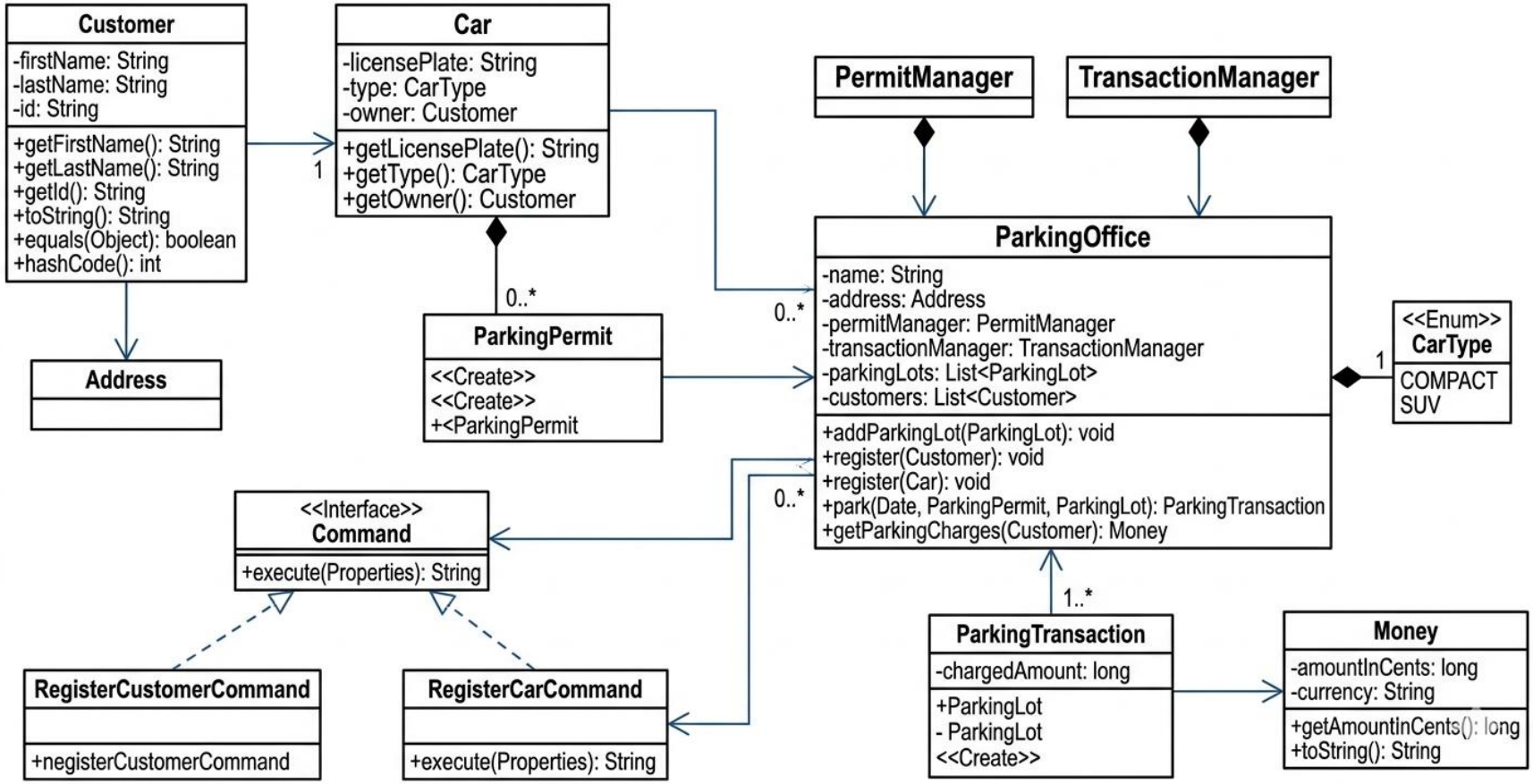
The core requirements involve maintaining a registry of unique customers and their associated vehicles, managing physical parking lot assets with varying rate structures, and accurately recording parking events (transactions). The system must differentiate between vehicle types (e.g., Compact vs. SUV) to apply specific pricing rules and provide a robust command-based interface for administrative actions. Data integrity is paramount, ensuring that every parked vehicle is linked to a valid permit and a verified owner.

UML Diagrams

The architecture follows the **Controller/Service Pattern**, where the ParkingOffice serves as the central hub, delegating specialized tasks to "Manager" classes.

- **Domain Layer:** Customer, Car, Address, ParkingPermit, ParkingLot, ParkingTransaction, Money.
- **Service Layer:** PermitManager, TransactionManager.
- **Controller Layer:** ParkingOffice, RegisterCustomerCommand, RegisterCarCommand.

Parking Management System - Class Diagram



Class Explanations and Implementation Notes

Core Domain Classes

- **Address:** A valued object storing geographic data. It is designed to be immutable once created to ensure data consistency across customer and office records.
- **Customer:** Represents the system user. It holds a unique ID and contact information.
    - *Implementation Note:* We utilized equals() and hashCode() overrides based on the ID to allow for efficient searching within Collections.
- **Car & CarType:** The Car class maintains the link between a license plate and an owner. The CarType (Enum) is critical for the pricing engine, distinguishing between COMPACT and SUV.
- **Money:** Encapsulates currency and amount (stored in cents) to avoid floating-point rounding errors during financial calculations.

Management Classes (The "Managers")

- **PermitManager:** Acts as the registry for all authorizations. It uses a HashMap<String, ParkingPermit> for lookup speed when a vehicle enters a lot.
- **TransactionManager:** The system's ledger. It records every parking event and provides filtering logic to retrieve history for specific customers.
  Coordinator Class
- **ParkingOffice:** The "Information Expert." It coordinates between the managers. For example, when the park() method is called, the Office fetches the permit from the PermitManager and then instructs the TransactionManager to record the event.

Controller/Command Classes

- **RegisterCustomerCommand & RegisterCarCommand:** These implement a Command interface. they parse raw Properties input, validate the data, and trigger the appropriate logic in the ParkingOffice.

Survey of Unit Tests

Testing was conducted using **JUnit 5**, focusing on both state and behavior:

Test Class	Focus Area	Key Assertions
AddressTest	Data Integrity	Verified that getters return exact constructor inputs.
CarTest	Association	Confirmed the Car correctly references its customer owner.
PermitManagerTest	Lifecycle	Verified that registering a car generates a valid, future-dated permit.
TransactionManagerTest	Logic/Filtering	Ensured that transactions can be filtered by a specific Customer ID.
CommandTests	Robustness	Tested "Sad Path" scenarios, such as missing IDs or null input properties.

Challenges and lessons learned

Challenges

The primary challenge I encountered was **Positioning and Mapping**. With several classes using multiple String parameters (First Name, Last Name, ID, Phone), we initially experienced "Parameter Swapping" where data was assigned to the wrong fields.

Secondly, handling all tests passing within RegisterCustomerCommandTest (3 tests passed, 1 test caused an error) and RegisterCarCommandTest (4 tests passed, 1 test failed) has been a challenge for me.

Lessons Learned

1. **Defensive Programming:** Implementing "Null Guards" at the Command level prevented NullPointerExceptions when the UI provided incomplete data.
2. **Encapsulation:** We learned that returning memory references (hex codes) instead of String IDs is a common sign of failing to access object state via proper getters.
3. **Enum Utility:** Using Enums for CarType eliminated "Magic String" errors and simplified the logic for applying the 20% SUV surcharge.
4. **Requirement Mapping:** During the integration phase, a discrepancy was noted where the billing total ($20.00) did not match the expected surcharged total ($24.00). Investigation revealed that the pricing logic in the ParkingLot class was applying a discount to COMPACT vehicles rather than the required surcharge to SUV vehicles. By refactoring the getDailyRate method to explicitly handle the SUV case with a 1.2 multiplier, the system successfully aligned with the project's financial requirements.

Project Configuration (pom.xml)

This project standardizes **Java 26** (LTS) for modern language features and **JUnit 5** for testing.

```
<project>
  <properties>
    <maven.compiler.source>26</maven.compiler.source>
    <maven.compiler.target>26</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <version>5.10.3</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Execution and Output

The application is tested via ParkingSystemApplication class (Main method). Below is the trace of the ParkingSystemApplication execution.

System Output:

```
=== Initializing Parking Management System ===
System Ready. Simulating Administrative Actions...

[Action: CUSTOMER] Result: CUST-999
[Action: CAR] Result: PRMT-ED2CF

=== Recording First Parking Event ===
- Stay 1 recorded in North Lot ($12.00)

=== Recording Second Parking Event ===
- Stay 2 recorded in North Lot ($12.00)

=== Final Billing Report ===
Total Charges for Customer CUST-999: $24.00
=== Simulation Complete ===
```